

Web Basic

---

# 비동기 처리

Promise

# Promise란?

```
let name = "Elice";

setTimeout(() => {
  name = "Hellobit";
}, 2000);

console.log(name); // 'Elice'
```

기본적인 비동기 처리는 처리 결과를 활용할 수 없다.

(콜 스택이 빈 후에 콜백 함수가 호출되기 때문)



# Promise

```
try {  
  setTimeout(() => {  
    // 에러 발생  
    console.log("doing something wrong");  
  }, 2000);  
} catch (err) {  
  console.log("에러 발생", err);  
}
```

또한 에러 처리(추후 학습)를 정상적으로 할 수도 없다.

(당장은 에러가 발생하지 않기 때문)





# Promise

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    console.log("Hello Elice");  
  }, 1000);  
});
```

Promise를 이용하면 앞선 문제를 모두 해결하며  
비동기 처리를 진행할 수 있다.

# Promise 생성

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    console.log("Hello Elice");  
  }, 1000);  
});
```

‘new Promise(콜백 함수)’와 같이 new 키워드를 통해 Promise 객체를 생성한다.

이때 콜백 함수가 무조건 인수로 넘겨져야 한다.



# Promise – resolve, reject

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    console.log("Hello Elice");  
  }, 1000);  
});
```

콜백 함수에는 `resolve`와 `reject` 함수가 인수로 전달된다.





# Promise – resolve, reject

```
const promise = new Promise((resolve, reject) => {  
  const fetchedData = fetchData();  
  
  if (fetchedData >= 5) resolve(fetchedData);  
  else reject("fetch failed");  
});
```

`resolve`: 비동기 처리의 성공을 나타냄

`reject`: 비동기 처리의 실패를 나타냄

# Promise 상태



# Promise의 상태

| 상태        | 설명                 |
|-----------|--------------------|
| pending   | 비동기 처리가 완료되기 전, 대기 |
| fulfilled | 비동기 처리가 완료되었으며, 성공 |
| rejected  | 비동기 처리가 완료되었으며, 실패 |

Promise는 처리 상태에 따라 세 가지 중 하나의 상태를 가진다.



# Promise – resolve, reject

```
const promise = new Promise((resolve, reject) => {  
  const fetchedData = fetchData();  
  
  if (fetchedData >= 5) resolve(fetchedData);  
  else reject("fetch failed");  
});
```

`resolve`: pending → fulfilled

`reject`: pending → rejected





# Promise – resolve, reject

```
> const promise = new Promise((resolve, reject) => {});  
< undefined  
> promise  
< ▶ Promise {<pending>}
```

Promise 객체 생성 직후에는 pending 상태



# Promise – resolve, reject

```
> function fetchData() { return 10; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
});

< undefined
> promise
< ▶ Promise {<fulfilled>: 10}
```

resolve가 실행되어 fulfilled인 상태



# Promise – resolve, reject

```
> function fetchData() { return 3; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
});

< undefined

✖ ▶ Uncaught (in promise) fetch failed

> promise

< ▶ Promise {<rejected>: 'fetch failed'}
```

reject가 실행되어 rejected인 상태

# Promise 처리



# Promise – then, catch, finally

| 메서드                                                  | 설명                       |
|------------------------------------------------------|--------------------------|
| <code>.then(r =&gt; { ... }, e =&gt; { ... })</code> | 비동기 성공 및 실패 후 처리         |
| <code>.catch(e =&gt; { ... })</code>                 | 비동기 실패 후 처리              |
| <code>.finally(() =&gt; { ... })</code>              | 비동기 성공/실패 여부 관계 없이 이후 처리 |

Promise 객체는 위 세 가지 메서드를 가지며,  
이 메서드를 사용해서 비동기 처리 완료 이후의 처리를 진행한다.  
각 메서드는 모두 Promise 객체를 반환한다.



# Promise – then

```
> function fetchData() { return 10; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).then(
  (result) => { console.log(`result is ${result}`); },
  (error) => { console.log(`error occurred by ${error}`); }
);

result is 10
< undefined
```

‘then’ 메서드에는 두 가지 콜백 함수가 인수로 전달된다.

1: 비동기 성공 시 처리 (인수: resolve 함수 호출 시 넘긴 값)

2: 비동기 실패 시 처리 (인수: reject 함수 호출 시 넘긴 값)



# Promise – catch

```
> function fetchData() { return 3; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).catch(
  (error) => { console.log(`error occurred by ${error}`); }
);

error occurred by fetch failed
< undefined
```

‘catch’ 메서드에는 비동기 실패 시 처리를 위한 콜백 함수가 인수로 전달된다.

(인수: reject 함수 호출 시 넘긴 값)



# Promise – finally

```
> function fetchData() { return 3; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).finally(
  () => { console.log("Promise finished"); }
);
```

Promise finished

< undefined

✖ Uncaught (in promise) fetch failed

‘finally’ 메서드에는 비동기 성공/실패 여부와 관계없이 실행될 콜백 함수가 인수로 전달된다.





# Promise Chaining

```
> function fetchData() { return 10; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).then(
  (result) => { console.log(`result is ${result}`); }
).catch(
  (error) => { console.log(`error occurred by ${error}`); }
).finally(
  () => { console.log("Promise finished"); }
);

result is 10
Promise finished
< undefined
```

보통 `then`, `catch`, `finally`는 메서드 체이닝\*을 통해 같이 사용된다.

(각 메서드는 Promise를 반환하기 때문에 체이닝이 가능하다.)

# Promise 메서드



# Promise.resolve

```
const resolvedPromise = Promise.resolve("Hello Elice");  
// 아래 코드와 동일하다:  
const resolvedPromise = new Promise((resolve, reject) => resolve("Hello Elice"));  
  
resolvedPromise.then((result) => { console.log(result); }); // 'Hello Elice'
```

Promise.resolve 메서드를 사용하면

인수로 전달받은 값을 resolve하는 프로미스를 반환 받을 수 있다.



# Promise.reject

```
const rejectedPromise = Promise.reject("Elice Rejected");  
// 아래 코드와 동일하다:  
const rejectedPromise = new Promise((resolve, reject) => reject("Elice Rejected"));  
  
rejectedPromise.catch((error) => { console.log(`error: ${error}`); }); // 'error: Elice Rejected'
```

‘Promise.reject’ 메서드를 사용하면  
인수로 전달받은 값을 reject하는 프로미스를 반환 받을 수 있다.



```
// 비동기 작업을 시뮬레이션하는 함수
function fetchData(url, delay) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(`Data from ${url}`);
    }, delay);
  });
}
```

```
// Promise.all을 사용하여 모든 비동기 작업이 완료될 때까지 기다림
Promise.all([
  fetchData('https://api.example.com/data1', 1000),
  fetchData('https://api.example.com/data2', 2000),
  fetchData('https://api.example.com/data3', 1500)
])
  .then((results) => {
    console.log('All promises resolved:');
    results.forEach((result, index) => {
      console.log(`Result ${index + 1}: ${result}`);
    });
  })
  .catch((error) => {
    console.error('One of the promises failed:', error);
  });
```

‘Promise.all’ 메서드를 사용하면  
여러 Promise를 병렬적으로 처리할 수 있다.  
(인수로 여러 Promise 객체가 담긴 배열 전달)

```
// 비동기 작업을 시뮬레이션하는 함수
function fetchData(url, delay) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(`Data from ${url}`);
    }, delay);
  });
}
```

```
// Promise.all을 사용하여 모든 비동기 작업이 완료될 때까지 기다림
Promise.all([
  fetchData('https://api.example.com/data1', 1000),
  fetchData('https://api.example.com/data2', 2000),
  fetchData('https://api.example.com/data3', 1500)
])
  .then((results) => {
    console.log('All promises resolved:');
    results.forEach((result, index) => {
      console.log(`Result ${index + 1}: ${result}`);
    });
  })
  .catch((error) => {
    console.error('One of the promises failed:', error);
  });
```

인수로 전달된 모든 Promise가 resolve된 후에 then으로 넘어간다.

이때 각 Promise의 결과는 배열의 형태로 전달된다.



# Promise.all – 실패 시 동작 (Fail-Fast)

```
Promise.all([
  fetchData('https://api.example.com/data1', 1000),
  Promise.reject('data2 실패!'), // ← 하나만 실패해도
  fetchData('https://api.example.com/data3', 1500)
])
  .then((results) => {
    console.log('All resolved:', results);
  })
  .catch((error) => {
    console.error('One failed:', error); // 'data2 실패!'
  });
```

**= Fail-Fast (빠른 실패)**

▶ One failed: data2 실패!

인수로 전달된 Promise 중 하나라도 reject되면,  
나머지 결과를 기다리지 않고 즉시 전체가 reject된다.

# Promise.all – 실패 시 동작 (Fail-Fast)

```
function fetchData(url, delay) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`Data from ${url}`), delay);
  });
}

Promise.all([
  fetchData('https://api.example.com/data1', 1000),
  Promise.reject(new Error('data2 fetch failed')),
  fetchData('https://api.example.com/data3', 1500)
])
  .then((results) => {
    console.log('All resolved:', results);
  })
  .catch((error) => {
    console.log('catch에 전달된 값:', error);
    console.log('타입:', typeof error);
    console.log('Error 객체인가?:', error instanceof Error);
  });
```

```
catch에 전달된 값: Error: data2 fetch failed
                  at <anonymous>:9:18
```

```
타입: object
```

```
Error 객체인가?: true
```

catch에는 가장 먼저 실패한 Promise의 reject 값이 전달된다.



## Promise.all – 실패 시 동작 (Fail-Fast)

```
// 케이스 1: 문자열로 reject
Promise.all([
  Promise.resolve(1),
  Promise.reject('단순 문자열 에러'),
])
.catch((error) => console.log('케이스1:', error));

// 케이스 2: Error 객체로 reject
Promise.all([
  Promise.resolve(1),
  Promise.reject(new Error('Error 객체 에러')),
])
.catch((error) => console.log('케이스2:', error));
```

케이스1: 단순 문자열 에러

케이스2: Error: Error 객체 에러  
at <anonymous>:11:18

catch에는 가장 먼저 실패한 Promise의 reject 값이 전달된다.

# Promise.allSettled – 모두 끝날 때까지

```
function fetchData(url, delay) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`Data from ${url}`), delay);
  });
}

Promise.allSettled([
  fetchData('https://api.example.com/data1', 1000),
  Promise.reject('data2 실패!'),
  fetchData('https://api.example.com/data3', 1500)
]).then((results) => {
  results.forEach((result, i) => {
    if (result.status === 'fulfilled') {
      console.log(`${i+1}: 성공 -`, result.value);
    } else {
      console.log(`${i+1}: 실패 -`, result.reason);
    }
  });
});
```

```
1: 성공 - Data from https://api.example.com/data1
2: 실패 - data2 실패!
3: 성공 - Data from https://api.example.com/data3
```

Promise.allSettled 메서드를 사용하면 성공/실패 여부와 관계없이  
모든 Promise가 완료될 때까지 기다린다.

# Promise.allSettled – 모두 끝날 때까지

```
function fetchData(url, delay) {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve(`Data from ${url}`), delay);  
  });  
}
```

```
Promise.allSettled([  
  fetchData('https://api.example.com/data1', 1000),  
  Promise.reject('data2 실패!'),  
  fetchData('https://api.example.com/data3', 1500)  
]).then((results) => {  
  console.log(results);  
});
```

각 결과는 다음 두 형태 중 하나

**{ status: 'fulfilled', value: ... }**

**{ status: 'rejected', reason: ... }**

```
▼ 0:  
  status: "fulfilled"  
  value: "Data from https://api.example.com/data1"  
  ▶ [[Prototype]]: Object  
▶ 1: {status: 'rejected', reason: 'data2 실패!'}  
▶ 2: {status: 'fulfilled', value: 'Data from https://api.example.com/data3'}  
  length: 3  
▶ [[Prototype]]: Array(0)
```

# Promise.all vs Promise.allSettled

|         | Promise.all                                    | Promise.allSettled    |
|---------|------------------------------------------------|-----------------------|
| 하나 실패 시 | 즉시 전체 reject                                   | 끝까지 기다림               |
| 반환값     | 성공 값들의 배열                                      | status 객체들의 배열        |
| 사용 시점   | 모두 성공해야 의미 있을 때<br>(모든 작업이 독립적이고 병렬 실행이 가능할 때) | 일부 실패를 허용할 때          |
| 동작 방식   | Fail-Fast                                      | Wait-All (모두 완료까지 대기) |



# 마이크로태스크 큐

# 마이크로태스크 큐 (Microtask Queue)

```
console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

console.log("2. 스크립트 종료");
```

위 코드는 어떤 순서로 실행될까?

# 마이크로태스크 큐 (Microtask Queue)

```
console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

console.log("2. 스크립트 종료");
```

앞서 배운 비동기 동작 원리를 고려하면

1 → 2 → 4 → 3의 순서로 예상해 볼 수 있다.

# 마이크로태스크 큐 (Microtask Queue)

```
> console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

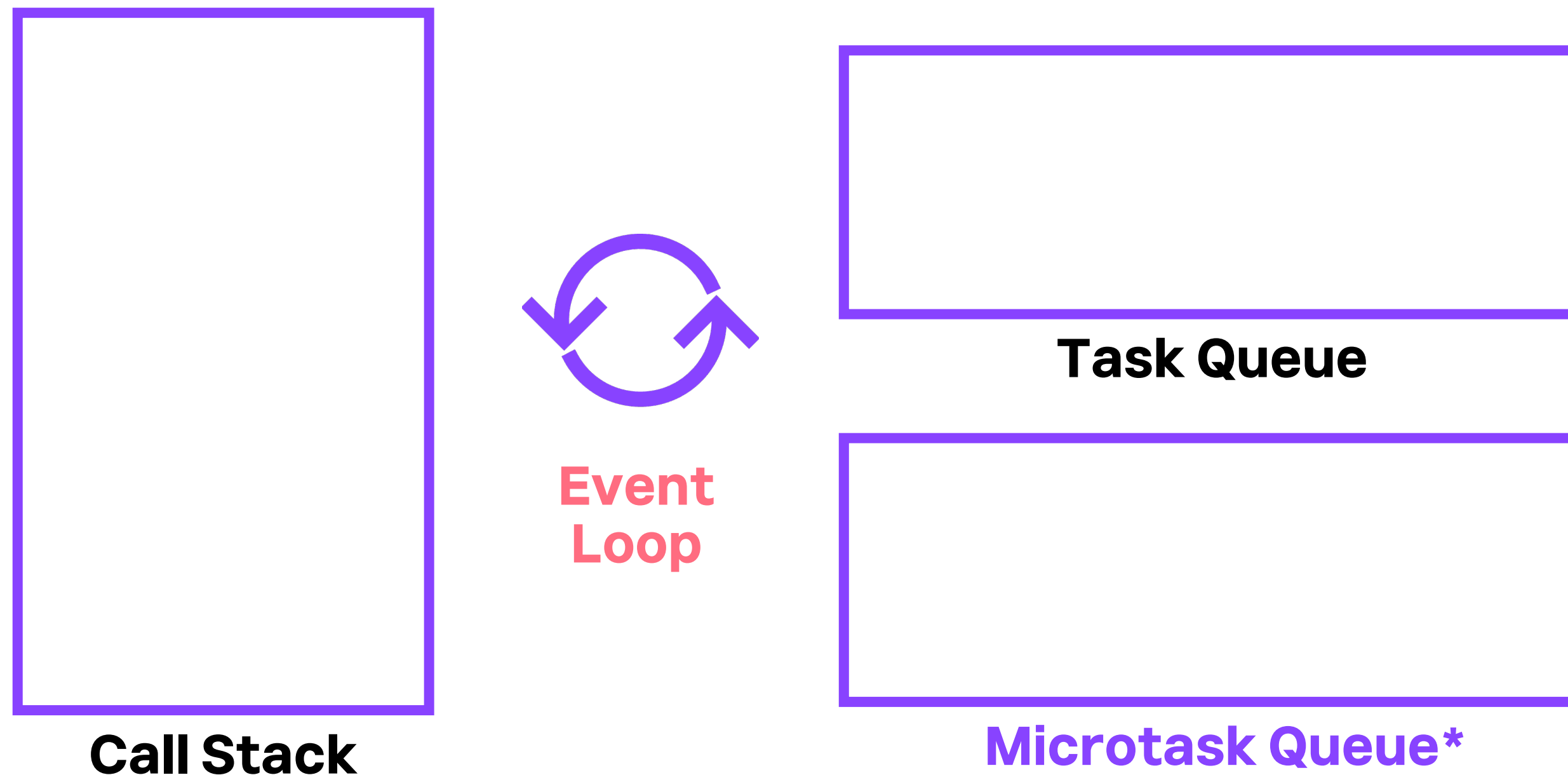
console.log("2. 스크립트 종료");
1. 스크립트 시작
2. 스크립트 종료
3. Promise 콜백 실행
< undefined
4. setTimeout 콜백 실행
```

그러나 실제로는 1 → 2 → 3 → 4의 순서로 실행된다.

왜 이런 것일까?



# 마이크로태스크 큐 (Microtask Queue)

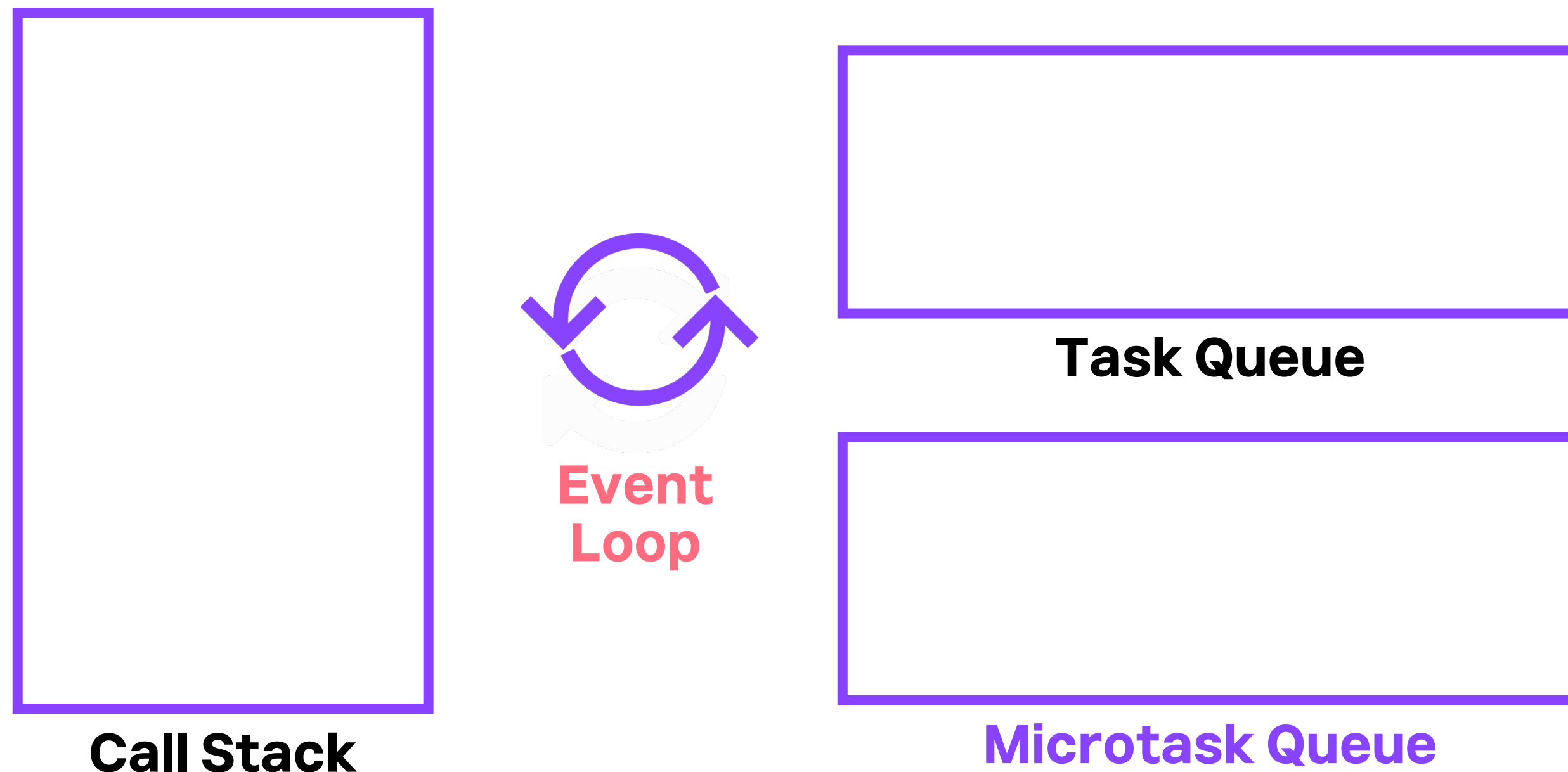


브라우저에는 마이크로태스크 큐라는 공간이 하나 더 존재한다.

마이크로태스크 큐는 태스크 큐보다 우선순위가 높다!

\*Microtask Queue는 Job Queue라고도 한다.

# 마이크로태스크 큐 (Microtask Queue)



즉, 마이크로태스크 큐에서 대기중인 작업이 먼저 콜 스택으로 들어가고,  
마이크로태스크 큐가 비면 태스크 큐의 작업이 콜 스택으로 들어간다.

# 마이크로태스크 큐 (Microtask Queue)

| 종류        | 저장하는 비동기 처리 작업     |
|-----------|--------------------|
| 마이크로태스크 큐 | Promise, Ajax      |
| 태스크 큐     | 타이머 함수, 이벤트 핸들러 함수 |

# 마이크로태스크 큐 (Microtask Queue)

```
console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

console.log("2. 스크립트 종료");
```

Promise는 마이크로태스크 큐에 저장되므로,  
setTimeout의 콜백 함수보다 먼저 실행된다.